

Exercício Prático 1

Aluno: Silas Hoffmann Reis Viana

Parte 1

- 1) Qual o problema de tempo associado a esse tipo de somador (pense no carry), considere o atraso médio de cada porta lógica de 10 ns.

O principal problema de tempo no somador ripple-carry é a propagação sequencial do carry: cada estágio depende do carry anterior. Assim, o bit mais significativo e o carry final só ficam prontos após a propagação pelos estágios anteriores.

- 2) Qual o tempo necessário para a computação de uma soma e do vai um em um somador de 4 bits.

Considerando atraso de 10 ns por porta e somador completo implementado por 2 meias-somas + 1 OR:

- $c1 = a0.b0 + c0. (a0 \text{ xor } b0) = 3 \text{ portas} \rightarrow 30 \text{ ns}$
- cada carry seguinte adiciona 2 portas ($AND + OR$) = + 20 ns
- para 4 bits: $c4 = 30 + 3 * 20 = 90 \text{ ns}$
- soma mais lenta (s3): precisa de c3 (90 ns) + XOR final (10 ns) = 100 ns

Logo, em 4 bits, o carry final sai em 90 ns e o bit de soma mais lenta em 100 ns.

- 3) O que seria necessário para um somador de 32 bits?

Para 32 bits (mesma lógica):

- carry final: $c32 = 30 + 31 * 20 = 650 \text{ ns}$
- soma mais lenta (s31): $c31 + XOR = (30 + 30 * 20) + 10 = 640 \text{ ns}$

- 4) Considerando esses tempos acima, calcule a frequência de operação de um somador de 32 bits.

A frequência máxima de operação é aproximadamente o inverso do caminho crítico

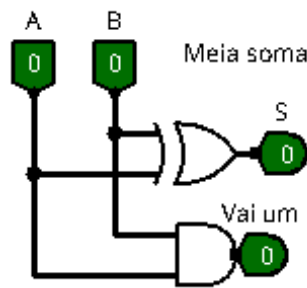
$$f_{max} \approx \frac{1}{650 \text{ ns}} = \frac{1}{650 \times 10^{-9}} \approx 1,54 \text{ MHz.}$$

- 5) Você consegue propor alguma forma de tornar essa soma mais veloz?

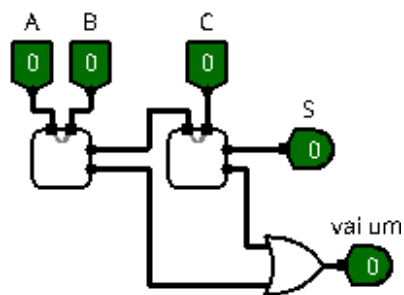
Para acelerar a soma, pode-se usar arquiteturas que reduzem a propagação em série do carry, como:

- carry-lookahead adder (CLA);
- somadores de prefixo (Kogge-Stone, Brent-Kung, etc.);
- carry-select adder.

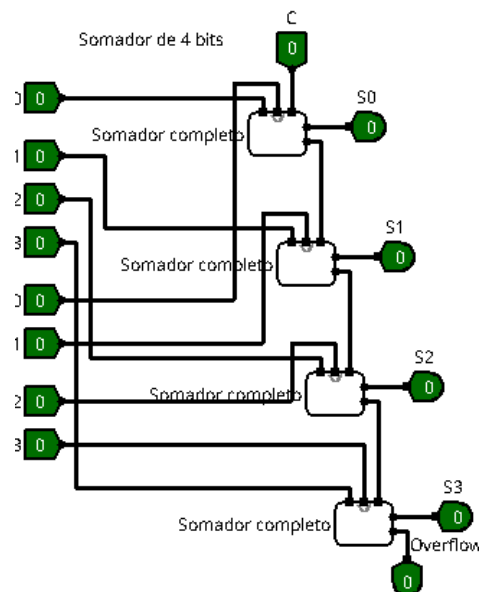
Essas soluções diminuem bastante o atraso em comparação ao ripple-carry.



Meia Soma

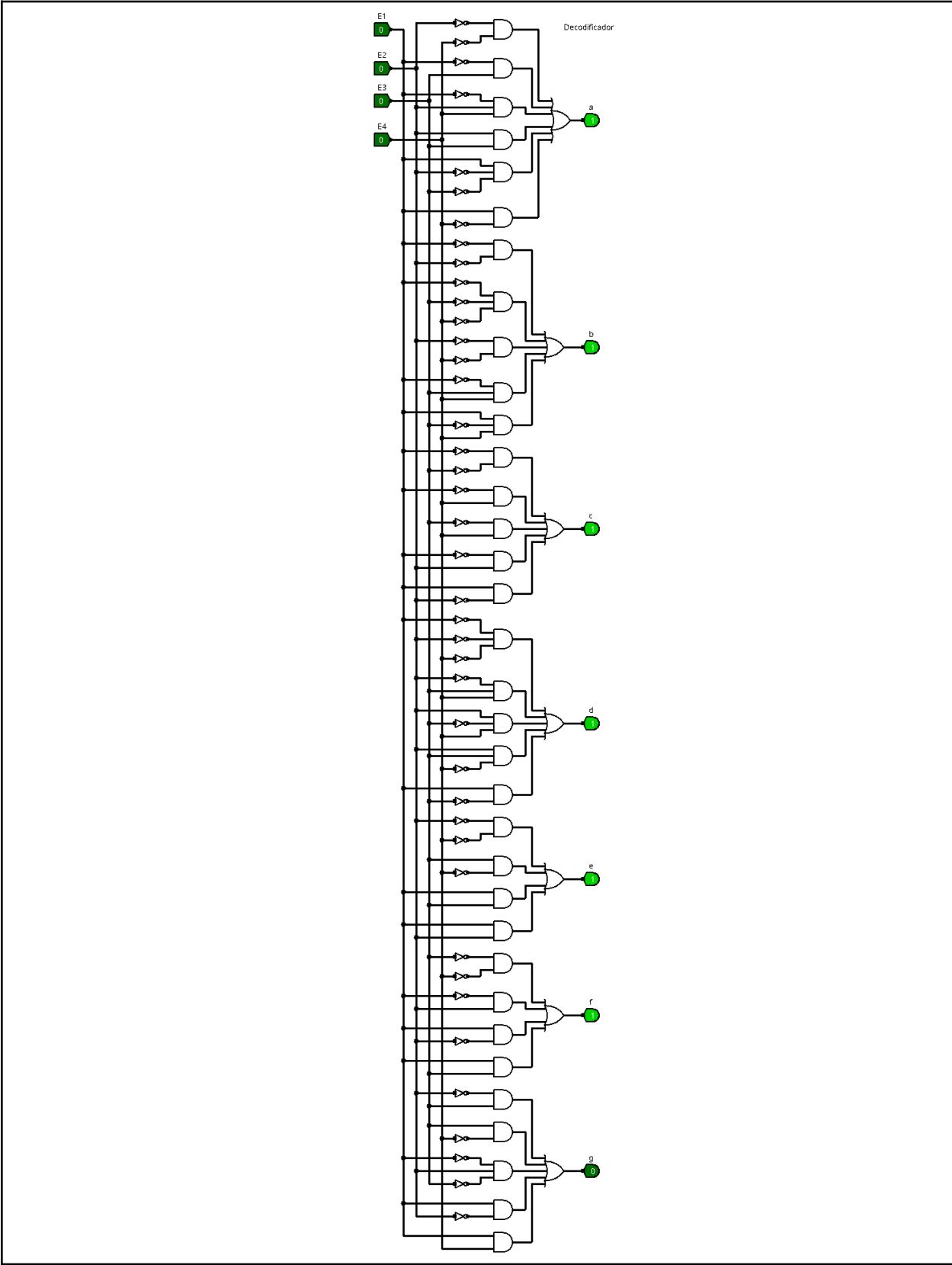


Somador Completo

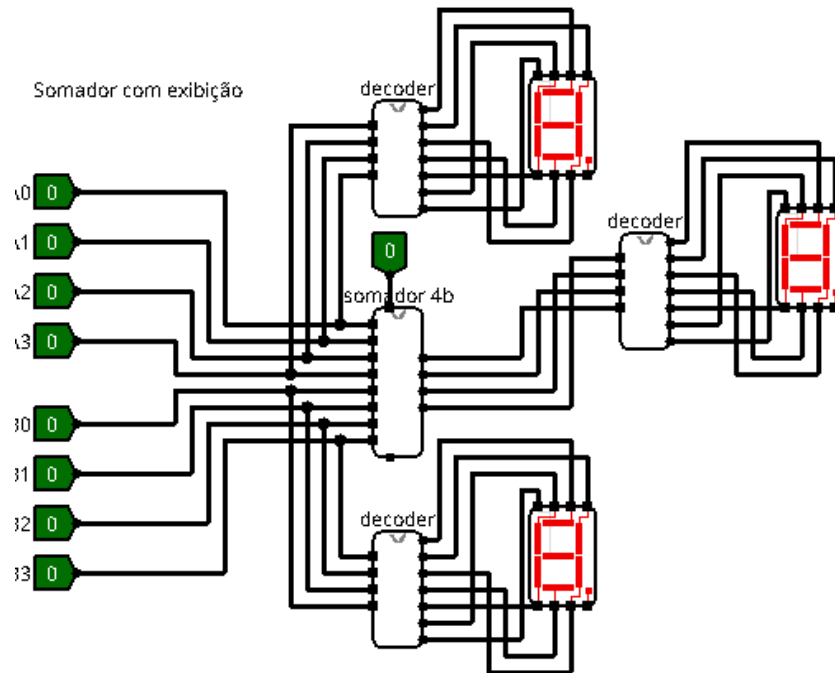


Somador de 4 Bits

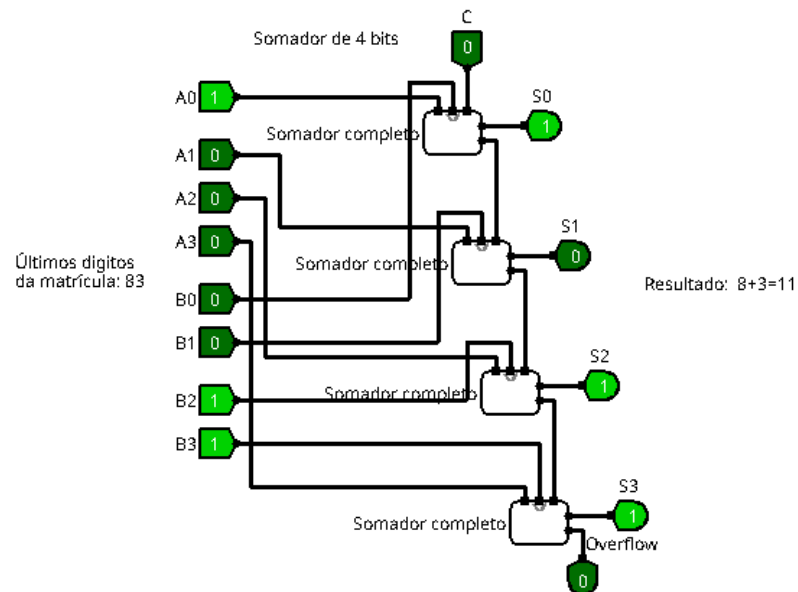
Parte 2



Decodificador



Somador com Exibição



Teste de execução do somador de 4 bits

Parte 3

A tabela utilizada para gerar os resultados segue uma sequência de 0 a F, porém a soma com números negativos, utilizando essa tabela, se torna problemática. A abordagem da imagem abaixo foi uma tentativa de manter a ordem atual e calcular os complementos, mas analisando os tempos de execução, ele possui um grande problema de performance, portanto uma nova abordagem é necessária para fazer os cálculos.

Se utilizarmos a tabela de complemento de dois, aproveitando da circularidade do circuito, o cálculo pode ser realizado corretamente com números positivos e negativos, diminuindo a quantidade de números positivos que podem ser representados com a mesma quantidade de bits

Representação Simples		Complemento de dois	
0	0000	0	0000
1	0001	1	0001
2	0010	2	0010
3	0011	3	0011
4	0100	4	0100
5	0101	5	0101
6	0110	6	0110
7	0111	7	0111
8	1000	-8	1000
9	1001	-7	1001
A	1010	-6	1010
B	1011	-5	1011
C	1100	-4	1100
D	1101	-3	1101
E	1110	-2	1110
F	1111	-1	1111

Observe que o seu somador de 4 bits possui a capacidade de também subtrair dois números, basta que a notação complemento de dois seja utilizada.

Considerando que os números são de 4 bits, apresente a soma dos seguintes valores:

- a) $+ 5 + (- 6) = - 1$
- b) $- 5 + (- 5) = - 10$
- c) $- 7 + (- 4) = - 11$
- d) $+ 7 + 4 = + 11$

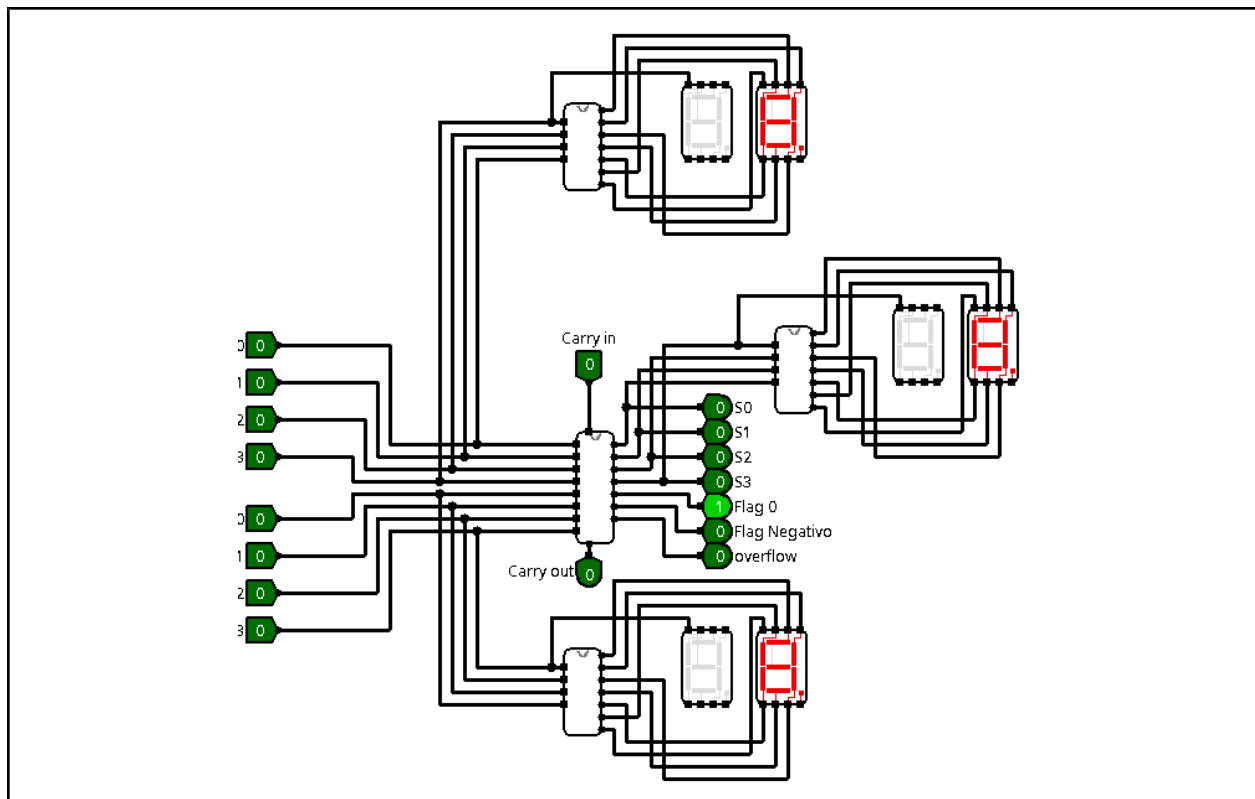
1) Porque as duas últimas operações irão ligar o overflow?

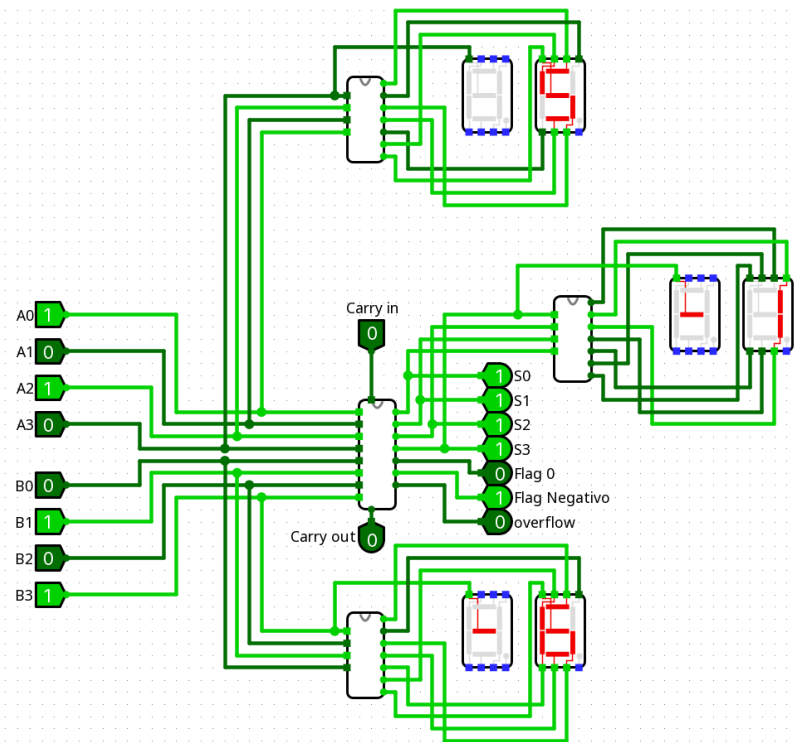
O somador é limitado entre -8 e 7, então os valores -11 e 11 que deveriam ser representados nas últimas operações, não podem ser representados. Sendo assim, a flag Overflow é acionada para indicar um erro na operação. Por outro lado, um cálculo pode ser aplicado para identificar o real resultado, utilizando o Overflow:

- Se a saída for > 7 , deve-se fazer Saída - 16, para conseguir o valor verdadeiro da saída (Exemplo: $- 7 + (- 4) = - 11 \rightarrow - 11 + 16 = + 5$)
- Se a saída for $< - 8$, deve-se fazer Saída + 16, para conseguir o valor verdadeiro da saída (Exemplo: $7 + 4 = + 11 \rightarrow + 11 - 16 = - 5$)

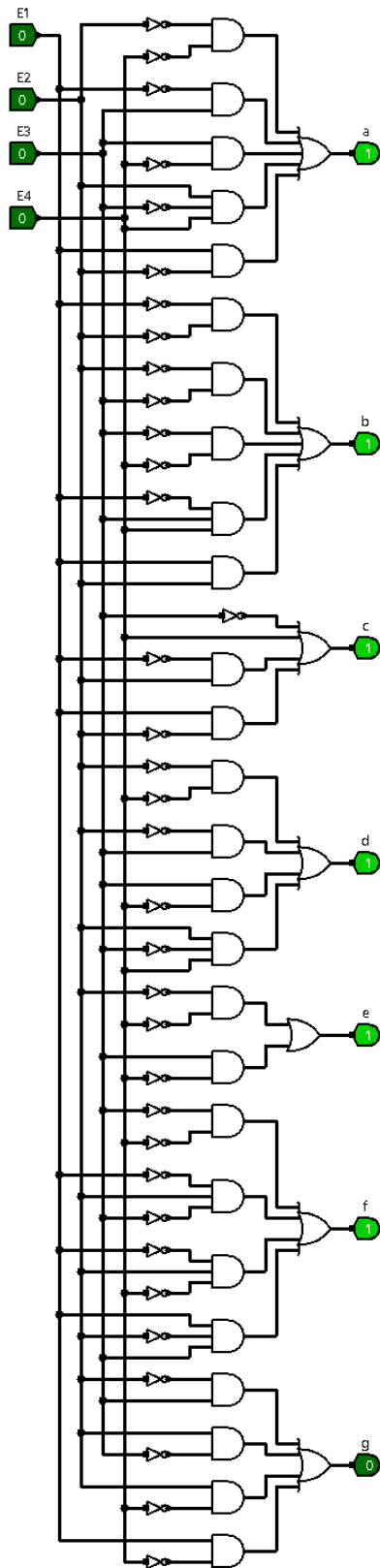
2) Qual a porta lógica que você utilizou para esta operação de identificar o overflow?

Para identificar o Overflow, foi utilizado uma porta XOR, com entradas CO2 e CO3





Somador de 4 bits com complemento de 2



Decodificador para complemento de 2

Parte 4

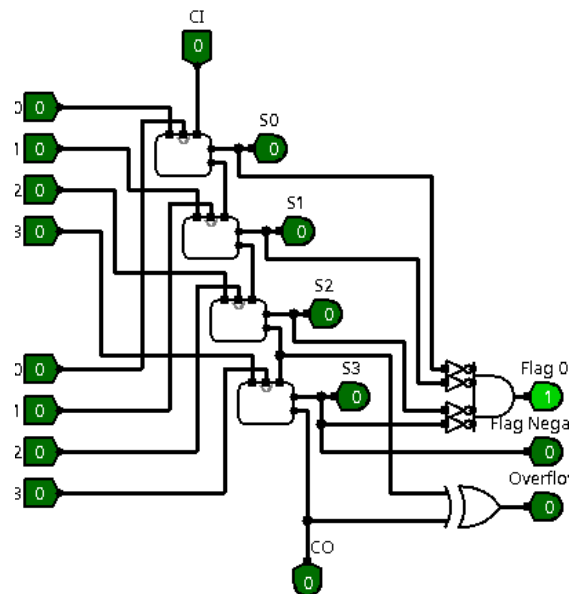
Normalmente os somadores possuem Flags para indicar através de 1 bit apenas o estado da saída. Implemente dois flags de 1 bit (apenas uma saída de 1 bit para cada flag) que indiquem:

- **Flag 0** - Quando todos os bits de saída do somador forem 0 (zero)
- **Flag Negativo** - Quando a saída for um número negativo

Você consegue pensar em alguma situação onde esses flags possam ser utilizados?

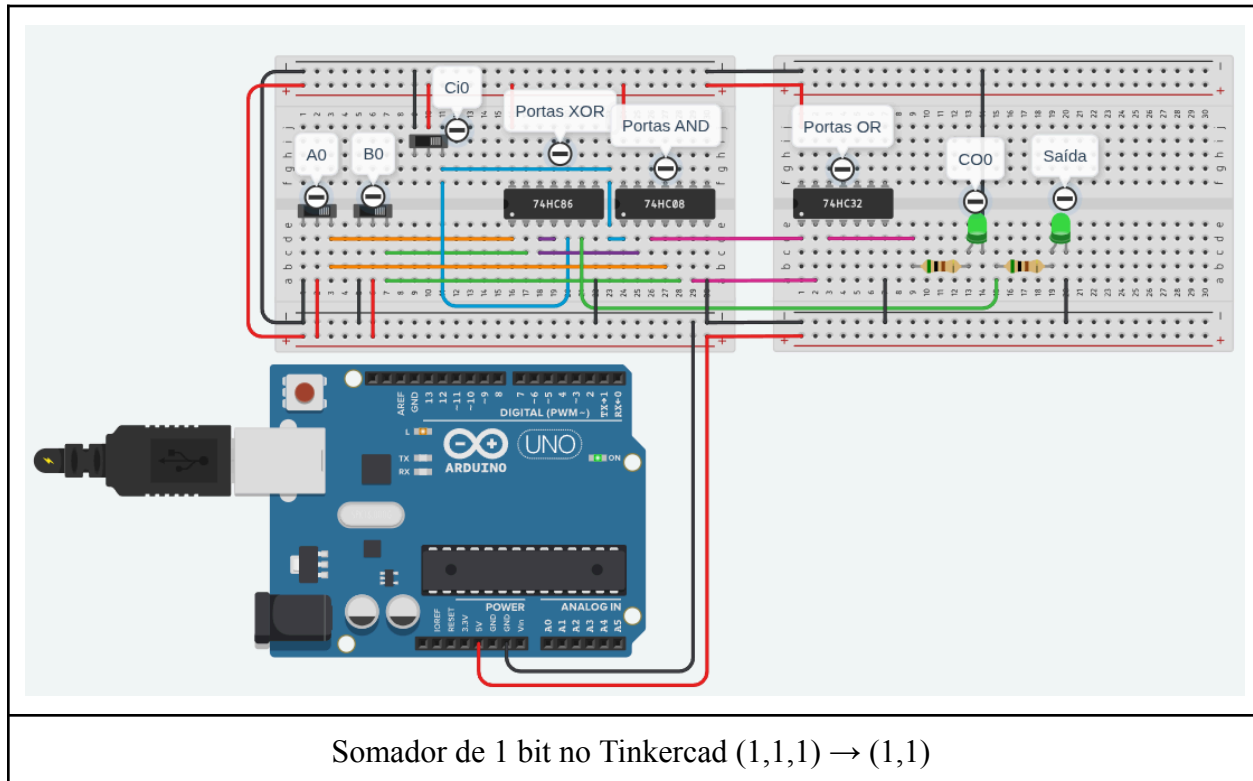
A flag 0 pode ser usada para comparar se os dois valores de entrada são iguais, enquanto a flag negativo pode indicar se A é menor que B, portanto essas saídas podem ser usadas para saber as diferenças entre as entradas, de forma simplificada:

- Flag Negativo = false & Flag 0 = true → valores iguais
- Flag Negativo = true → A menor que B
- Flag Negativo = false & Flag 0 = false → A maior que B



Somador de 4 bits com flags extras

Parte 5



Notas: O arduino da imagem só foi utilizado como fonte, mas não foi utilizado nenhuma codificação. Como cada módulo de operação possui 4 entradas A, 4 entradas B e 4 saídas, utilizei mais de um conjunto de portas em cada módulo, para evitar a repetição de módulos, desnecessariamente.

Link do circuito:

https://www.tinkercad.com/things/clgLZiWlj0r-somador-de-1-bit?sharecode=5i60E_zY4s3ePhSYbZi5BffdIVCgi4VzTr6UejLjvdM

Parte 6

Somador de 1 bit em Arduino

```
const int pinoCo = 13;
const int pinoSaida = 10;

int portaAnd(int a, int b) {
    return a & b;
}

int portaOr(int a, int b) {
    return a | b;
}

int portaXor(int a, int b) {
    return a ^ b;
}

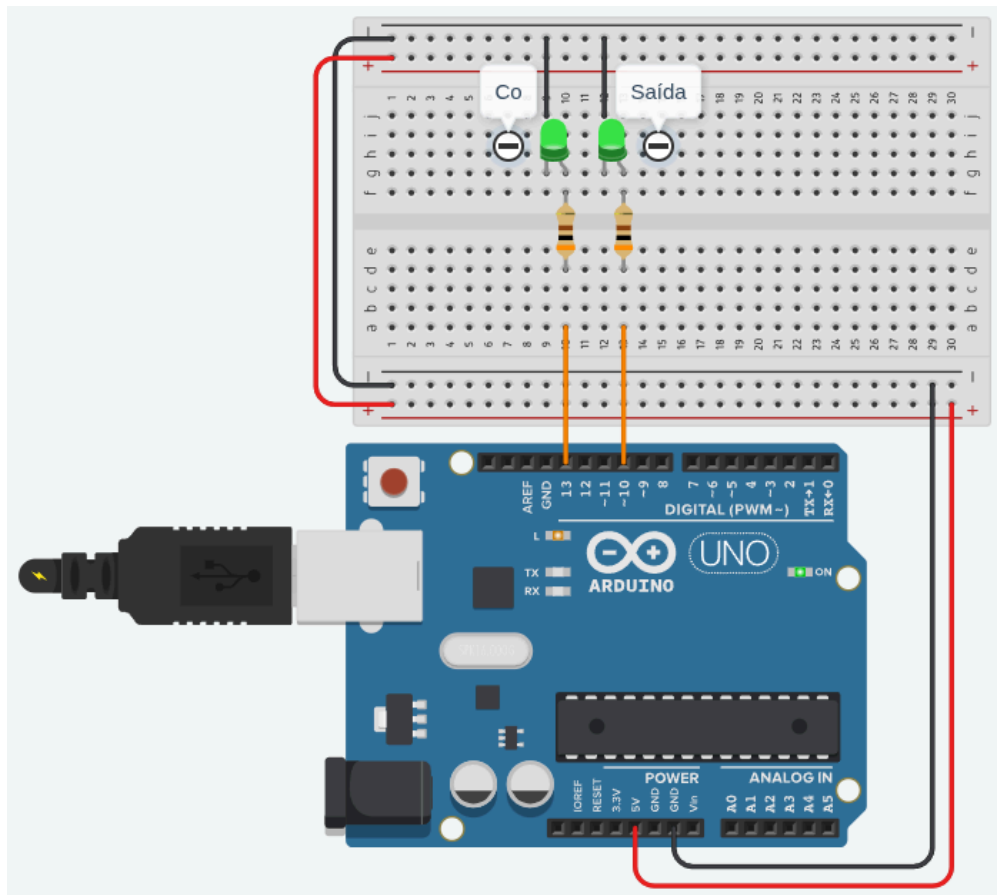
void setup() {
    Serial.begin(9600);
    pinMode(pinoCo, OUTPUT);
    pinMode(pinoSaida, OUTPUT);
    Serial.println("Digite as entradas no formato  
A B Ci (ex: 1 0 1). Para interromper, digite  
-1");
}

void loop() {
    if (Serial.available() > 0) {
        int A = Serial.parseInt();
        if (A < 0) {
            Serial.println("Sistema interrompido.");
            digitalWrite(pinoCo, LOW);
            digitalWrite(pinoSaida, LOW);
            while (true) {}
        }
        int B = Serial.parseInt();
        int Ci = Serial.parseInt();
        int xor1 = portaXor(A, B);
        int and1 = portaAnd(A, B);
        int and2 = portaAnd(xor1, Ci);
        int saida = portaXor(xor1, Ci);
        int Co = portaOr(and1, and2);

        digitalWrite(pinoCo, Co);
        digitalWrite(pinoSaida, saida);

        Serial.print("Entradas: ");
        Serial.print(A);
        Serial.print(" ");
        Serial.print(B);
        Serial.print(" ");
        Serial.println(Ci);
        Serial.print("Co: ");
        Serial.print(Co);
        Serial.print(" | Saida: ");
        Serial.println(saida);
    }
}
```

Código do somador de 1 bit em C++



Monitor serial

Digite as entradas no formato A B Ci (ex: 1 0 1). Para interromper, digite -1
 Entradas: 1 1 1
 Co: 1 | Saída: 1

Circuito do somador de 1 bit em Arduino

Link do circuito:

<https://www.tinkercad.com/things/lkMnPIE2KXu-somador-de-1-bit-em-c?sharecode=ntIqHBT0oltsW-qO9cSXF2j6aOsb3qcsxFREukS6psk>